# Pacotes e Encapsulamento

## Orientação a Objetos – DCC025

2012.3

# Pacotes

- Maneira de organizar classes em grupos
- Um pacote contém um número qualquer de classes relacionadas de acordo com:
  - Propósito
  - Escopo
  - Herança

# Pacotes - Utilidades

- Permitem organizar classes em *unidades*, de forma similiar à organização de arquivos em pastas, facilitando a sua manipulação.

- Reduzem problemas de conflitos de nomes, pois classes podem ser "escondidas" em pacotes.

# Pacotes - Utilidades

- Permitem proteger classes, atributos e métodos de forma mais abrangente que proteção baseada em classes (modificadores de visibilidade *package* e *protected*).

- Podem ser usados para identificação de classes (um pacote de classes para um certo propósito pode conter o nome da organização e o nome do autor do pacote).

# Pacotes -hierarquia

- Um pacote pode conter outro pacote, definindo uma hierarquia de pacotes.

- Exemplo:
  - O pacote *java*, contém os pacotes *io*, *net*, *util* e *awt*.
  - O pacote *awt*, por sua vez, contém o pacote *image* e outros. Assim o nome completo deste último pacote é *java.awt.image*.

# Pacotes - utilização

- Classes do pacote *java.lang* (*System*, *Date*, *String*, *Integer, ...*) são **automaticamente disponíveis**.
- Uma classe de outro pacote pode ser referenciada através de seu nome completo (por exemplo, *java.awt.Font f = new java.awt.Font();* ).
- Classes de outro pacote **usadas com freqüência** devem ser "**importadas**" (individual ou coletivamente), permitindo que sejam referenciadas somente por nome (por exemplo, *Font*).

# Pacotes – Comando import

- Usado para importar uma certa classe. Ex:

  *import java.util.Vector* ;

  …

  *Vector v = new Vector();*

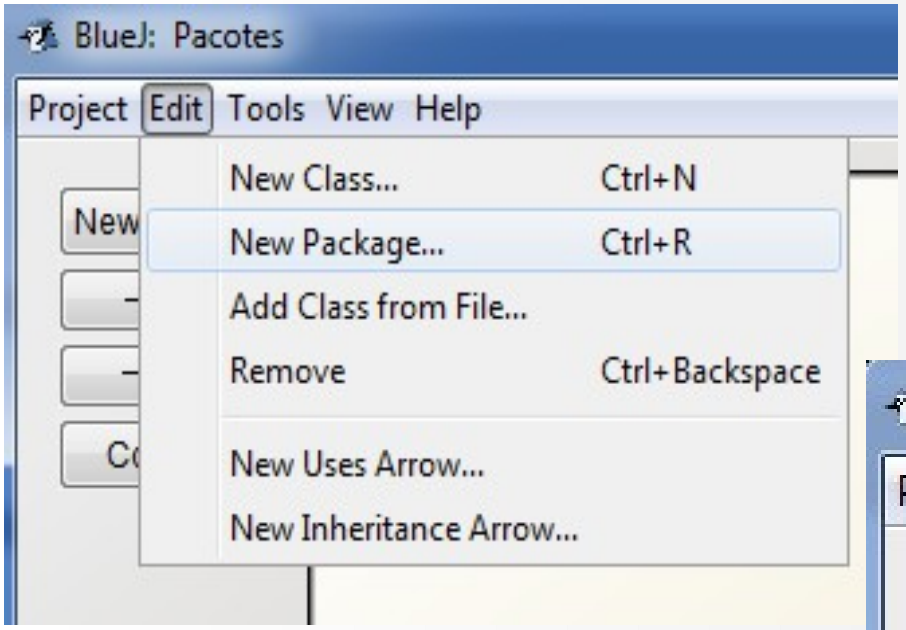
- Usado para importar todas as classes **públicas** de um pacote. Ex: *import java.awt.* ;*

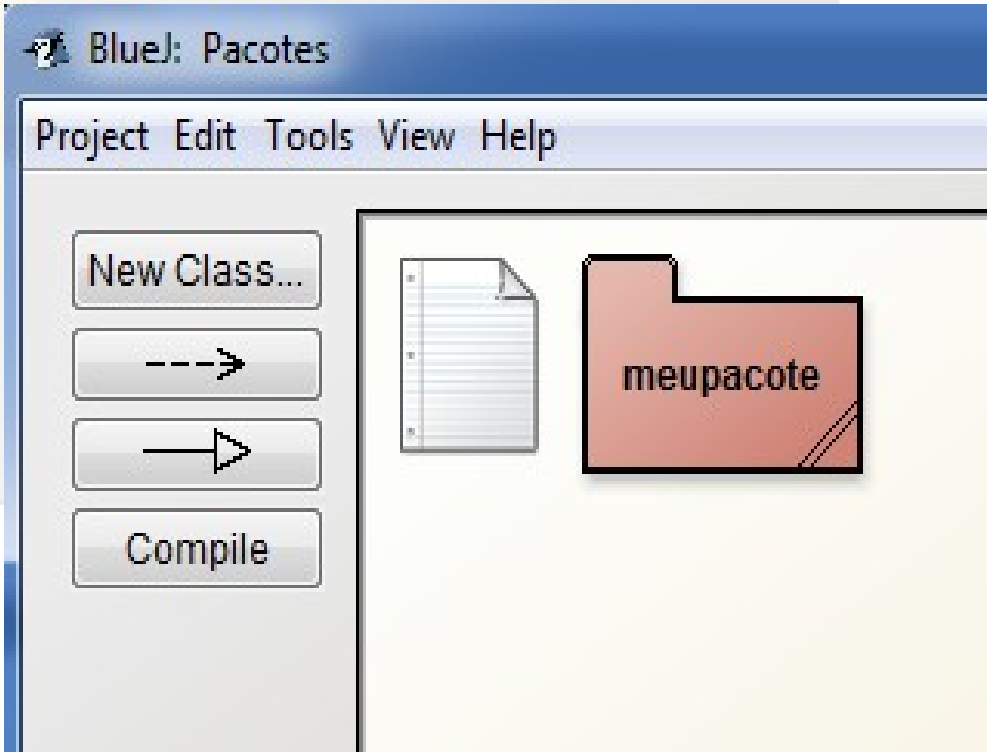
- Não importa pacotes recursivamente. Por exemplo, *import java.awt.** não importa as classes do pacote *image*.
- Devem aparecer no início de um arquivo.

# Pacotes - Prática

- Usar o comando *package* para inserir classes em um pacote.
  - Deve ser único em um arquivo e deve ser o primeiro comando presente no arquivo.
  - Todas as classes definidas no arquivo passam a fazer parte do pacote especificado.
  - Quando não especificado, as classes são inseridas no pacote *default* de Java.
  - Pode ser especificado com o mesmo nome de pacote em diversos arquivos.

# Pacotes - BlueJ



## Pacotes

- Uma classe só é visível fora de seu pacote se for qualificada como *public*.
- Um arquivo pode conter diversas classes, mas somente uma delas pode ser qualificada como *public* (o nome do arquivo deve ser igual ao nome dessa classe).
- Um comando *import* do tipo *import p.** importa somente as classes públicas do pacote *p*.

# Encapsulamento

# Encapsulamento - Idéia

Este gato representa um objeto da classe Gato, e possui altura como característica. Todo e qualquer gato não pode possuir altura inferior a 9.

O que pode acontecer se não usarmos encapsulamento?

# Encapsulamento - Idéia

```
3  public class Gato {
4
5      public int altura;
6
7  }
```

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Gato g1 = new Gato();
8          g1.altura = 0;
9      }
10 }
```

Péssimo. Não deve-se deixar isso acontecer.
Como evitar?

Que implicações isso gera?

# Encapsulamento - Idéia

```
3  public class Gato {
4
5      public int altura;
6
7  }
```

Mudar para

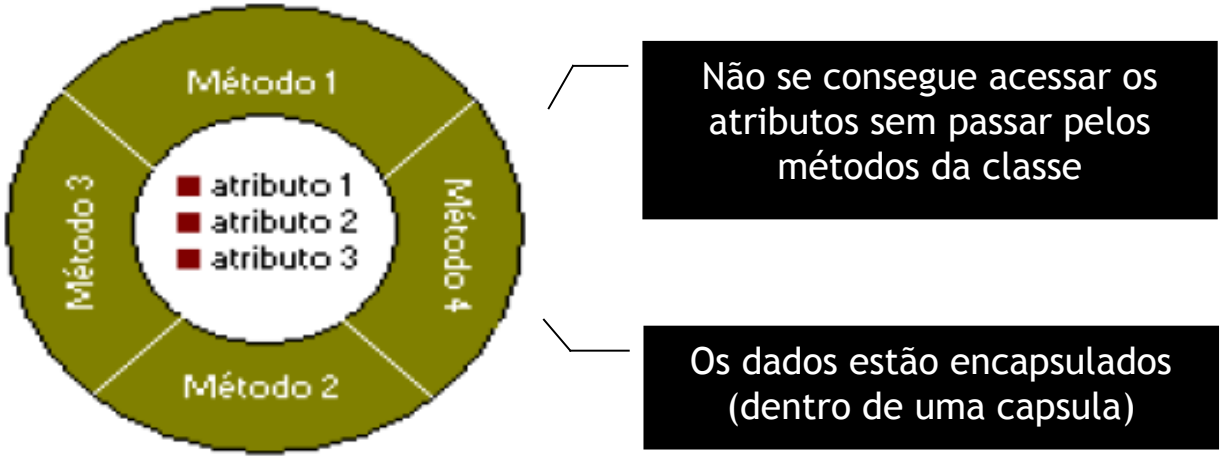
```
3  public class Gato {
4
5      private int altura;
6
7      public void setAltura(int altura){
8          if(altura >= 9){
9              this.altura = altura;
10         }
11     }
12 }
```

```
3  public class Principal {
4
5      public static void main(String args[]){
6
7          Gato g1 = new Gato();
8          g1.setAltura(8);
9      }
10 }
```

Óbvio que a alteração para 8 não será executada. Existe um método para impedir isto.

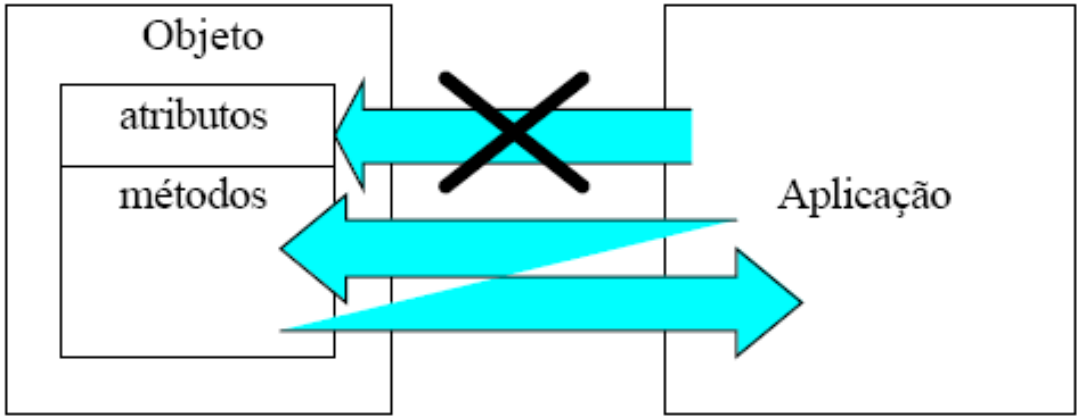
# Encapsulamento

- Definição
  - Empacotamento dos atributos e métodos numa mesma classe. Isto protege os dados contra corrupção, pois somente os métodos da classe poderão alterar as estruturas de dados da classe.



# Encapsulamento

- Definição
  - O conceito de encapsulamento é decorrente do fato de se combinar os dados (atributos) e o código que manipula estes dados (métodos) em um único Objeto. Ele garante que a única forma de acesso aos dados é através dos métodos disponíveis ao usuário (chamados públicos). Os demais métodos e os atributos da classe ficam sendo privados.



# Encapsulamento

- Definição
  - Permite que certas características ou propriedades dos objetos possam ou não ser vistas ou modificadas externamente.
  - A idéia básica relacionada ao encapsulamento é o de ocultamento de informações.

# Encapsulamento

- Vantagens
  - Esconder a complexidade do Código
    - Esconder detalhes de implementação
  - Proteção dos dados
    - São acessados somente via métodos da classe
    - Evita-se que sejam corrompidos por aplicações externas
  - Pode-se alterar a implementação do objeto sem alterar a do cliente
    - Objeto a: oferece serviços a outros objetos
    - Objeto b: utiliza serviços oferecidos por a
    - Objeto b é dito ser cliente de a.
    - Pode-se alterar a estrutura interna de “a” sem alterar a estrutura de “b”

# Encapsulamento

- Observação
  - Encapsulamento garante que a minha classe seja uma **caixinha preta** para o usuário: ele não sabe o que há dentro do objeto, sabe apenas para que ele serve e quais os métodos disponíveis para a manipulação deste.
  - Contudo...
    - Se, ao projetarmos uma classe, não fornecemos métodos de acesso adequados, teremos dificuldades em criar aplicações eficientes com objetos instanciados da mesma.

# Exemplificação

```
class Funcionario {
    double salario;
}
```

O atributo SALARIO de um objeto da classe FUNCIONARIO pode ser acessado ou modificado
por código escrito em qualquer classe ...

a) Do mesmo pacote da Classe Funcionário
b) Externa à Classe Funcionário

# Exemplificação

```
class Funcionario {
    private double salario;

    void aumentaSalario(double aumento) {
        // lógica para aumentar o salário
    }
}
```

Incorporando encapsulamento

A variável “salário” passa a ser privada com a declaração private. Isso faz com que somente métodos internos dessa classe possam manipular seus valores

Se algum código fora da classe Funcionário tentar acessar ou alterar o valor do atributo privado Salario de um objeto dessa Classe, um erro de compilação é gerado

# Exemplificação

Classe Teste

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Funcionario func = new Funcionario();
6
7          func.salario = 100;
8          System.out.println(func.salario);
9      }
10 }
11
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> Teste [Java Application] C:\Program Files\Java\jdk1.6.0_21\jre\bin\jav
100.0
```

Classe Salário

```
1 abstract public class Funcionario {
2
3     public double salario;
4
5     public void aumentarSalario(int aumento){
6         this.salario = salario + (salario * aumento);
7     }
8
9 }
```

# Exemplificação

Classe Teste

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Funcionario func = new Funcionario();
6
7          func.salario = 100;
8          System.out.println(func.salario);
9      }
10 }
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> Teste [Java Application] C:\Program Files\Java\jdk1.6.0_21\jre\bin\javaw.exe (28/03/2011 09:41:08)
Exception in thread "main" java.lang.Error: Unresolved compilation problems:
        The field Funcionario.salario is not visible
        The field Funcionario.salario is not visible

        at Teste.main(Teste.java:7)
```

Classe Salário

```
1  abstract public class Funcionario {
2
3      private double salario;
4
5      public void aumentarSalario(int aumento){
6          this.salario = salario + (salario * aumento);
7      }
8  }
```

# Exemplificação

Quando salario é “private”

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Funcionario func = new Funcionario();
6
7          func.
8
```

- aumentarSalario(double aumento) : void - Funcionar
- equals(Object obj) : boolean - Object
- getClass() : Class<?> - Object
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object
- toString() : String - Object
- wait() : void - Object
- wait(long timeout) : void - Object
- wait(long timeout, int nanos) : void - Object

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Funcionario func = new Funcionario();
6
7          func.
8
9      }
10 }
11
```

- salario : double - Funcionario
- aumentarSalario(double aumento) : void - Funcionar
- equals(Object obj) : boolean - Object
- getClass() : Class<?> - Object
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object
- toString() : String - Object
- wait() : void - Object
- wait(long timeout) : void - Object
- wait(long timeout, int nanos) : void - Object

Problems | @ Javadoc

Quando salario é “public”

# Encapsulando

```java
abstract public class Funcionario {

    private double salario;

    public void aumentarSalario(int aumento){
        this.salario = salario + (salario * aumento);
    }

    public double imprimir(){
        return this.salario;
    }
}
```

```java
public class Teste {

    public static void main(String args[]){

        Funcionario func = new Funcionario();

        System.out.println(func.imprimir());
        func.aumentarSalario(50);
        System.out.println(func.imprimir());
    }
}
```

Problems | Javadoc | Declaration | Console | Progress

```
<terminated> Teste [Java Application] C:\Program Files\Java\jdk1.6.0_21\jre\bin\java
100.0
150.0
```

# Encapsulando

Problema: não há métodos públicos na Classe Funcionário - não existe interface

```
1  public class Teste {
2
3      public static void main(String args[]){
4
5          Funcionario func = new Funcionario();
6
7          func.
8      }
9  }
10
```

- equals(Object obj) : boolean - Object
- getClass() : Class<?> - Object
- hashCode() : int - Object
- notify() : void - Object
- notifyAll() : void - Object
- toString() : String - Object
- wait() : void - Object
- wait(long timeout) : void - Object
- wait(long timeout, int nanos) : void - Object

```
1  abstract public class Funcionario {
2
3      private double salario;
4
5      private void aumentarSalario(int aumento){
6          this.salario = salario + (salario * aumento);
7      }
8
9      private double imprimir(){
10         return this.salario;
11     }
12 }
```

# Encapsulamento

- Reforçando
  - Definir todos os atributos como privados e definir métodos para implementar as lógicas de acesso e alteração é quase uma regra da orientação a objetos. O intuito é ter sempre um controle centralizado do dados dos objetos para facilitar a manutenção do sistema.

# Atributos Privados

- Acesso
  - Para permitir o acesso aos atributos privados de uma maneira controlada, a prática mais comum é criar dois métodos, um que retorna o valor e outro que muda o valor.
  - A convenção para esses métodos é de colocar a palavra get ou set antes do nome do atributo.

# Atributos Privados

Os método com “get” retornam o valor de um atributo, enquanto que aqueles com “set” realizam uma modificação em seu valor

```
public class Conta {

    private double saldo;
    private double limite;
    private Cliente titular;

    public double getSaldo() {
        return this.saldo;
    }

    public void setSaldo(double saldo) {
        this.saldo = saldo;
    }

    public double getLimite() {
        return this.limite;
    }

    public void setLimite(double limite) {
        this.limite = limite;
    }
```

# Uso de Get e Set

- Observação
  - É uma má prática criar uma classe e, logo em seguida, criar getters e setters para todos seus atributos.
  - Você só deve criar um get ou set se tiver a real necessidade.

# Encapsulamento – Vantagens

- Código mais fácil de modificar e manter
- Ajuda a controlar acesso
- Aumenta coesão em um módulo – tudo que é relacionado se encontra no mesmo lugar
- Ajuda a mudar uma parte do código sem afetar outras